Your part — Gaurav Rustagi (2024519071)

Project: Renewable Energy Grid Simulator (SDG 7 & 13). Full context is in `README.md` and the SRS if you want the whole picture, but everything you actually need is below.

Your area: "Simulation & Experience" — you own the dispatch logic and the dashboard people actually see and interact with during the demo.

Files you own

- `sim/engine.py` — the simulation engine (Reactive vs. Forecast-Driven dispatch)
- `app.py` — the Streamlit dashboard
- `test_engine.py` — the regression check for the above

What you need to be able to explain, unprompted

The dispatch rule. Reactive: battery reacts to the current hour only. Forecast-Driven: the battery holds back most of its charge when the Phase 2 AI model says a bigger demand peak is still a few hours out — spending a small shortfall on fossil now to save charge for the real peak, instead of spending it early.

The bug you'd get asked about first if anyone reads `DECISIONS.md` closely: the first version of this had Reactive and Forecast-Driven producing bit-for-bit *identical* annual results. Root cause: redistributing *when* the battery discharges can't change the *total* energy balance over a full year — whatever's held back now just gets discharged later, so nothing is actually saved in aggregate. Fixed by modelling a real Indian DISCOM Time-of-Day tariff (6pm-10pm = costlier, dirtier diesel-peaker backup; everything else = cheaper, cleaner baseload). Forecast-Driven genuinely shifts battery use toward that expensive/dirty window — same total energy, real cost/CO2 win.

Why renewable % barely changes between modes but cost/CO2 do — this is the single most likely question from your guide. Answer: see #2. Renewable % is a raw energy-balance number (near-fixed by design); cost/CO2 depend on *when* that energy gets used, which is exactly what the dispatch rule controls.

How the dashboard is wired: sliders (solar/wind/battery MW & MWh) → `cached_run()` → both modes run for the same capacity → side-by-side comparison at the top, detailed demand-vs-supply + battery-SoC charts at the bottom for whichever mode/week you pick.

The performance fix: the trained model file is ~45MB; naively reloading it from disk on every slider move took 5.7 seconds — way too slow for a live demo. Fixed by caching the loaded model in-process, so only the very first interaction after starting the server pays that cost; every one after is ~0.26 seconds.

The scale rescale: the grid was originally sized at 60MW average demand (small-city scale), which made the dashboard's Rupee figures balloon into the billions — visibly contradicting the "microgrid" framing from the pitch. Rescaled to 5MW (genuine campus/community microgrid scale), regenerated data, retrained the model, reverified everything.

Your remaining tasks

- [ ] Write the report's **Simulation Engine** + **Dashboard** sections
- [ ] Own the **live demo delivery** — you're the one clicking sliders in front of the guide. Rehearse `DEMO_SCRIPT.md` end to end at least twice before the real thing, especially the "drop the battery to near-zero and watch the advantage collapse" move — it's the strongest honesty signal in the whole demo, don't skip it
- [ ] Take screenshots / a short screen recording of the dashboard for the report
- [ ] Run `test_engine.py` yourself once and read its output — you should be able to say what each of its 6 checks actually verifies, not just that it says "PASS"

Before you touch anything: environment setup

```
cd "C:\Users\Lenovo\OneDrive\Desktop\renewable-grid-simulator"
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python test_engine.py       # confirm it says 6/6 PASS on your machine
streamlit run app.py        # opens the dashboard in your browser
```

Questions? Ask Adwaith, or just ask directly — no need to guess

If anything above is unclear, or if you and Adwaith want to swap which half you're each taking, say so — the split isn't fixed, it's just a reasonable starting point.